Cyber Shujaa

Security Analyst

ASSIGNMENT REPORT

THM - Attactive Directory

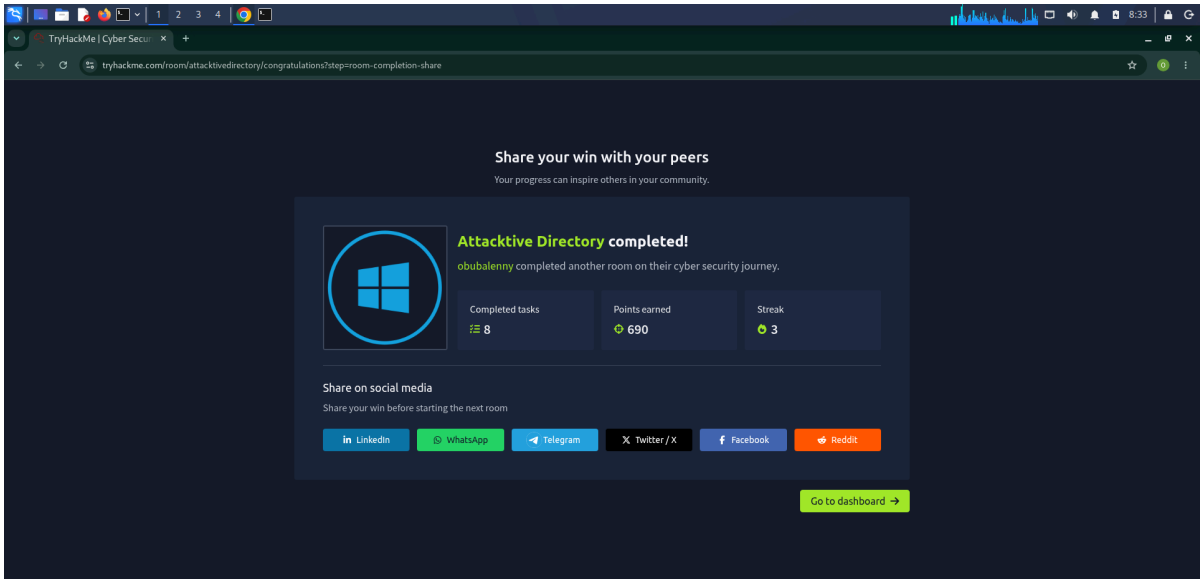
Full name	*Lenson Obuba*
Admission number	*cs-sa13-26017*
Email	*obubalenny@gmail.com*
Week and assignment	*Week 6, Assignment 2: Attactive Directory*
Platform	*TryHackMe*
Date submitted	*10/17/2026*

1. Module completion

Shareable link

https://tryhackme.com/room/attacktivedirectory/congratulations?step=room-completion-share

Completion screenshot



2. Introduction

This module focuses on the comprehensive compromise of a vulnerable Windows Domain Controller through the TryHackMe Attacktive Directory environment. For a security analyst, mastering Active Directory security is imperative, as it serves as the identity backbone for the vast majority of enterprise networks, making it a high-value target for threat actors. Going into this assignment, I expected to move past isolated service exploits and instead develop a unified, sequential methodology that links initial domain enumeration directly to credential harvesting and eventual domain-wide administrative control.

3. The work

3.1 Enumeration - Welcome to Attacktive Directory

What this section covered

This section covered the first stage of attacking the domain: using nmap to fingerprint the target as a domain controller, then using SMB/NetBIOS enumeration tools to extract information (domain name, SID) without any credentials at all. The main takeaway was that a correctly locked-down service (enumdomusers denied) can still leak the same information through a different, less-restricted channel (enum4linux/null session).

Question 1

What tool will allow us to enumerate port 139/445?

Answer

enum4linux

Question 2

What is the NetBIOS-Domain Name of the machine?

Answer

THM-AD

Question 3

What invalid TLD do people commonly use for their Active Directory Domain?

Answer

.local

How you got there

I started with nmap: nmap -sV -sC 10.48.177.148. This fingerprinted the host as a Windows domain controller for the domain spookysec.local (NetBIOS name THM-AD, hostname AttacktiveDirectory), with SMB (139/445), Kerberos (88), LDAP (389/3268), and WinRM (5985) all exposed.
I tested for an unauthenticated SMB session using rpcclient -U "" -N 10.48.177.148. The null session connected successfully, but running `enumdomusers` inside it returned NT_STATUS_ACCESS_DENIED, direct bulk user enumeration over that channel was blocked, which I noted as a dead end rather than assuming null sessions were unusable entirely.
I then ran enum4linux -a 10.48.177.148, the dedicated tool for enumerating SMB/NetBIOS services on 139/445. It confirmed the same null session, and despite also hitting NT_STATUS_ACCESS_DENIED on querydispinfo and enumdomusers, it successfully retrieved the domain SID (S-1-5-21-3591857110-2884097990-301047963) and the NetBIOS domain name (THM-AD) via a different RPC call (lsaquery) that wasn't locked down the same way. The domain's DNS name, spookysec.local, answered the third question directly: .local is a non-routable, invalid top-level domain that is nonetheless very commonly used for internal AD domains.

Evidence

Terminal output of the enum4linux -a run against the target, showing the null session, domain SID, and NetBIOS domain name.



Terminal output of the enum4linux -a run against the target, showing the null session, domain SID, and NetBIOS domain name.This proves an unauthenticated session was possible against the domain controller and that it leaked domain identity information before any credentials were obtained.

3.2 Enumerating Users via kerberos

What this section covered

This section moved from enumerating the domain itself to enumerating its user accounts, using Kerbrute to query the Kerberos service (port 88) directly. Kerberos will confirm or deny that a username exists before any password is checked, so a list of candidate names can be turned into a list of confirmed real accounts without needing valid credentials at all.

Question 1

What command within Kerbrute will allow us to enumerate valid usernames?

Answer

userenum

Question 2

What notable account is discovered? (These should jump out at you)

Answer

svc-admin

Question 3

What is the other notable account is discovered? (These should jump out at you)

Answer

backup

How you got there

I downloaded the room's provided user list and password list and installed Kerbrute (the prebuilt Linux binary

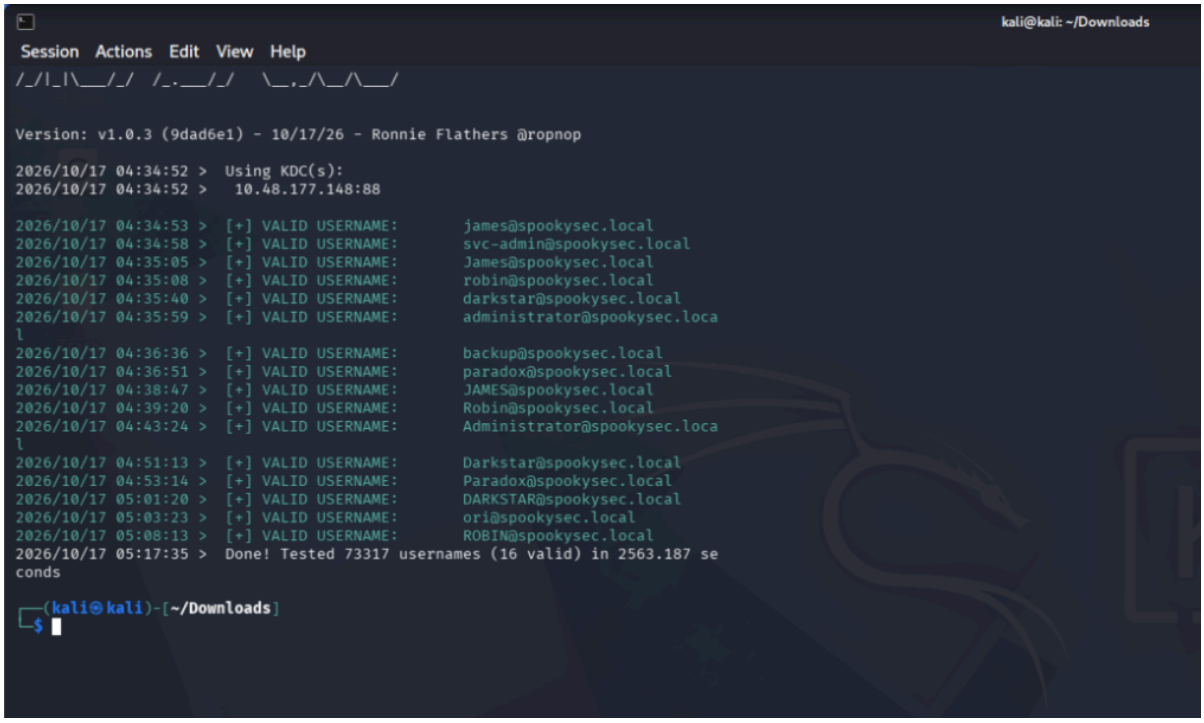
I ran: kerbrute userenum -d spookysec.local --dc 10.48.177.148 ~/Downloads/userlist.txt

It returned 16 hits, but several were just case variations of the same account (Windows account lookups aren't case-sensitive), giving 8 distinct real accounts: james, svc-admin, robin, darkstar, administrator, backup, paradox, and ori.

Of those, svc-admin and backup stand out immediately: unlike the other names in the list, these are functional/service accounts, and service and backup accounts are classic Active Directory weak points.

Evidence

What this screenshot shows: Terminal output of the kerbrute userenum run, showing the valid usernames returned by the Kerberos service



This proves that a list of confirmed, real domain accounts could be obtained purely by querying Kerberos, without ever authenticating.

3.3 Exploitation - Abusing Kerberos

What this section covered

This section exploited AS-REP roasting: when a domain account has "Does not require Kerberos pre-authentication" set, the KDC will hand back an encrypted authentication blob for that account without ever checking a password, and that blob can be cracked offline. Of the two accounts found, svc-admin had this setting and backup did not, so svc-admin was the target. Cracking the resulting hash against the room's password list turned it into a working username and password.

Question 1

We have two user accounts that we could potentially query a ticket from. Which user account can you query a ticket from with no password?

Answer

svc-admin

Question 2

Looking at the Hashcat Examples Wiki page, what type of Kerberos hash did we retrieve from the KDC? (Specify the full name)

Answer

Kerberos 5, etype 23, AS-REP

Question 3

What mode is the hash?

Answer

18200

Question 4

Now crack the hash with the modified password list provided, what is the user accounts password

Answer

management2005

How you got there

I used Impacket's GetNPUsers.py to request a Kerberos ticket for each of the two candidate accounts without supplying a password:
GetNPUsers.py spookysec.local/ -usersfile ~/vulnusers.txt -no-pass -dc-ip 10.48.177.148
The KDC returned a full AS-REP hash for svc-admin ($krb5asrep$23$svc-admin@SPOOKYSEC.LOCAL:...), while it explicitly reported that backup "doesn't have UF_DONT_REQUIRE_PREAUTH set" and refused the same request, a clean confirmation of which account was actually vulnerable, and a ruled-out dead end worth recording rather than ignoring.
The hash prefix $krb5asrep$23$ identifies it on Hashcat's examples wiki as "Kerberos 5, etype 23, AS-REP", hashcat mode 18200.
I saved the hash to a file and first tried hashcat -m 18200 svc-admin.hash passwordlist.txt, but it failed immediately with an OpenCL/memory error, my laptop has no GPU and very limited RAM (8GB total), and hashcat's default "pure" kernels couldn't allocate enough memory on the CPU device. Rather than keep fighting hashcat's memory limits, I switched to John the Ripper, which is lighter on resources for CPU-only cracking:
john --format=krb5asrep --wordlist=passwordlist.txt svc-admin.hash
John cracked it almost instantly, recovering the password management2005 for svc-admin. This gave the first full, working domain credential: spookysec.local\svc-admin:management2005.

Evidence

Terminal output showing GetNPUsers.py returning the AS-REP hash for svc-admin, and John the Ripper cracking it to recover the plaintext password.



This proves a full, usable domain credential was obtained purely by abusing a Kerberos pre-authentication misconfiguration, with no prior access to the domain required.

3.4 Enumeration - Back to Basics

What this section covered

With a working domain credential (svc-admin) in hand, this section went back to basic SMB share enumeration, but now authenticated instead of anonymous. Authenticated access reveals far more than a null session does, in this case, a custom "backup" share that wasn't visible before, containing a credentials file for a second account.

Question 1

What utility can we use to map remote SMB shares?

Answer

smbclient

Question 2

Which option will list shares?

Answer

-L

The -L flag is what lists shares, and smbclient itself is the utility

```
┌──(kali㉿kali)-[~/Downloads]
└─$ smbclient -L //10.48.177.148/ -U svc-admin
Password for [WORKGROUP\svc-admin]:

        Sharename       Type      Comment
        ---------       ----      -------
        ADMIN$          Disk      Remote Admin
        backup          Disk
        C$              Disk      Default share
        IPC$            IPC       Remote IPC
        NETLOGON        Disk      Logon server share
        SYSVOL          Disk      Logon server share
```

Question 3

How many remote shares is the server listing?

Answer

6

Question 4

There is one particular share that we have access to that contains a text file. Which share is it?

Answer

backup

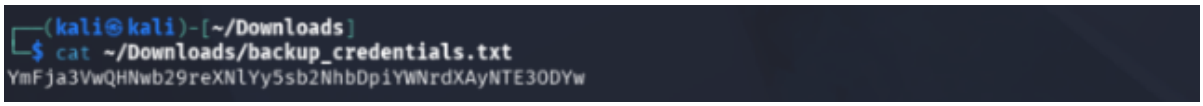
```
┌──(kali㉿kali)-[~/Downloads]
└─$ smbclient //10.48.177.148/backup -U svc-admin
Password for [WORKGROUP\svc-admin]:
Try "help" to get a list of possible commands.
smb: \> ls
  .                                   D        0  Sat Apr  4 15:08:39 2020
  ..                                  D        0  Sat Apr  4 15:08:39 2020
  backup_credentials.txt              A       48  Sat Apr  4 15:08:53 2020
```

Question 5

What is the content of the file?

Answer

YmFja3VwQHNwb29reXNlYy5sb2NhbDpiYWNrdXAyNTE3ODYw



Question 6

Decoding the contents of the file, what is the full contents?

Answer

backup@spookysec.local:backup2517860



How you got there

I listed the shares available to an authenticated user with:smbclient -L //10.48.177.148/ -U svc-admin
This returned 6 shares: ADMIN$, backup, C$, IPC$, NETLOGON, and SYSVOL. The first four are Windows default/administrative shares that exist on every domain controller; backup stood out as the one custom share that didn't belong to that standard set, so I connected into it specifically:

smbclient //10.48.177.148/backup -U svc-admin
Listing its contents showed a single file, backup_credentials.txt. I downloaded it (get backup_credentials.txt) and read it, finding a single line of text that looked like base64 (a run of letters/numbers with no spaces and a length divisible by 4). Decoding it with `echo "string" | base64 -d revealed a second full credential pair: backup@spookysec.local:backup2517860. This gave a second working domain account, obtained purely through file access that only became possible once we had the first account's credentials, a clear example of how one compromised account creates visibility into resources that lead to further compromise.

Evidence

Terminal output of the authenticated smbclient share listing, the backup share's contents, and the base64-decoded credentials file.



This proves that authenticated SMB access exposed a share and a credentials file that were completely invisible during anonymous/null-session enumeration, yielding a second working domain account.

3.5 Domain Privilege Escalation - Elevating privileges within the domain

What this section covered

This was the final stage of the chain: turning two ordinary-looking user accounts into full control of the domain. The backup account had been granted "Replicating Directory Changes" and "Replicating Directory Changes All" rights, permissions normally reserved for domain controllers themselves, used to keep multiple DCs in sync. Holding those rights on a regular user account meant that account could ask the domain controller for any password hash in the domain, including the built-in Administrator's, without ever touching the NTDS.dit database file directly or needing local access to the server at all.

Question 1

What method allowed us to dump NTDS.DIT?

Answer

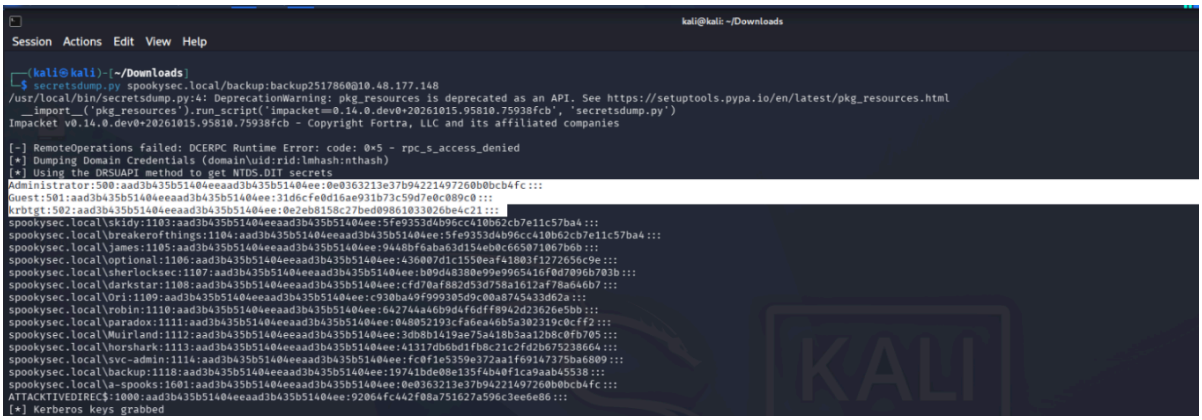
DRSUAPI

Question 2

What is the Administrators NTLM hash?

Answer

0e0363213e37b94221497260b0bcb4fc



Question 3

What method of attack could allow us to authenticate as the user without the password?

Answer

Pass the Hash

Question 4

Using a tool called Evil-WinRM what option will allow us to use a hash?

Answer

-H

How you got there

Using the backup account's credentials (backup:backup2517860, I recovered in section 3.4) and its replication rights, I ran Impacket's secretsdump.py against the domain controller:
secretsdump.py spookysec.local/backup:backup2517860@10.48.177.148
This is the DCSync technique: secretsdump used the DRSUAPI method to impersonate a domain controller and request directory replication data from the real DC, which handed back the NTLM hash (and Kerberos keys) for every account in the domain, including built-in Administrator (0e0363213e37b94221497260b0bcb4fc) and every other user found earlier. Notably, the dump succeeded via DRSUAPI even though an initial RemoteOperations/registry-based attempt was denied (rpc_s_access_denied), another small dead end I discovered, since it confirms the backup account's privilege came specifically from replication rights, not general admin access to the box.
With the Administrator's NTLM hash in hand, there was no need to crack it into a plaintext password at all. Pass the hash is the technique of authenticating directly with a captured NTLM hash, since NTLM authentication only ever needs the hash, never the original password. I used Evil-WinRM, whose -H flag accepts a hash in place of a password:
evil-winrm -i 10.48.177.148 -u Administrator -H 0e0363213e37b94221497260b0bcb4fc
This opened an interactive PowerShell session as thm-ad\administrator, confirmed with whoami, full control of the domain, achieved entirely through credential abuse and misconfigured permissions, without ever needing to crack the Administrator's actual password.

Evidence

Terminal output of the secretsdump.py DCSync dump (showing the Administrator NTLM hash) and the Evil-WinRM pass-the-hash login confirming whoami as thm-ad\administrator.



This proves complete compromise of the Active Directory domain, achieved by chaining an open null session, Kerberos username enumeration, an AS-REP roast, an exposed credentials file, and a replication-rights abuse , with no step requiring a known password at the start.

3.6 Flag Submission

What this section covered

The final task asked for a flag from each compromised account's desktop: svc-admin, backup, and Administrator. The room suggested retrieving the user accounts' flags via RDP and the Administrator's via Evil-WinRM, but since the already-open Evil-WinRM session was authenticated as Administrator, which has filesystem access to every profile on the box , I was able to retrieve all three flags from that single session.

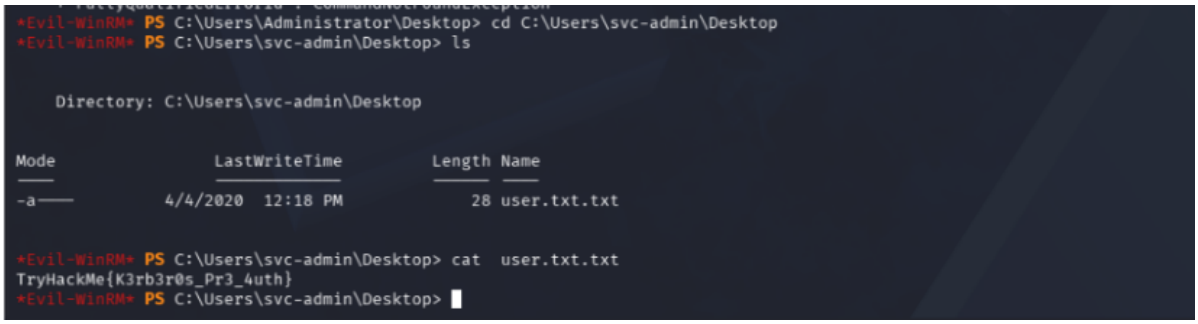
Task 1

Svc-admin - Flag

SVC-Admin User Flag: Located at C:\Users\svc-admin\Desktop\user.txt.txt

Answer

TryHackMe{K3rb3r0s_Pr3_4uth}



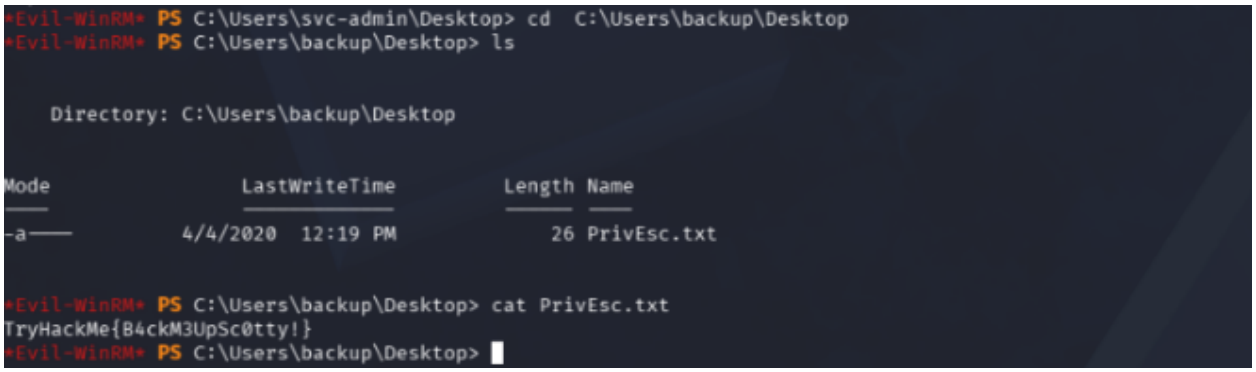
Task 2

backup - Flag

Backup User Flag: Located at C:\Users\backup\Desktop\PrivESC.txt

Answer

TryHackMe{B4ckM3UpSc0tty!}



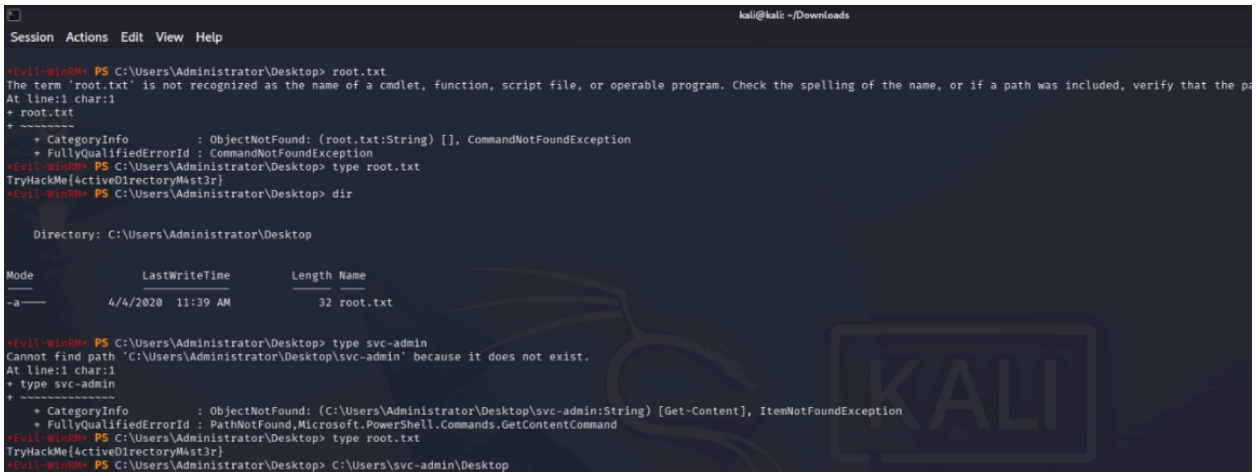
Task 3

Administrator - Flag

Administrator User Flag: Located at C:\Users\Administrator\Desktop\root.txt

Answer

TryHackMe{4ctiveD1rectoryM4st3r}



How you got there

From the existing Evil-WinRM session (authenticated as Administrator via pass-the-hash), I first checked the Administrator's own Desktop:
cd C:\Users\Administrator\Desktop then dir, which showed root.txt. type root.txt gave TryHackMe{4ctiveD1rectoryM4st3r}.
cd C:\Users\svc-admin\Desktop then `ls`, which showed user.txt.txt. `cat user.txt.txt` gave TryHackMe{K3rb3r0s_Pr3_4uth}.
cd C:\Users\backup\Desktop then `ls`, which showed PrivEsc.txt. cat PrivEsc.txt gave TryHackMe{B4ckM3UpSc0tty!}.

Evidence

Terminal output of the Evil-WinRM session retrieving all three flag files from the Administrator, svc-admin, and backup desktops.



This proves all three accounts' flags were recovered, completing the room's objective and demonstrating that Administrator-level access reaches every account's data on the domain controller.

4. Challenges and how you resolved them

Several technical roadblocks provided valuable lessons in tool selection and workflow efficiency during this room. The full 65,535-port nmap scan was heavily throttled over the VPN, prompting a switch to a fast top-1000 port scan so progress could continue while the comprehensive scan ran in the background. Dependency issues also arose when Kali Linux's PEP 668 environment blocked plain pip installs for Impacket, which was resolved using the --break-system-packages flag, an accepted workaround for dedicated penetration testing VMs. Furthermore, when hashcat failed to crack the AS-REP hash due to strict CPU memory allocation limits on a laptop without a dedicated GPU, switching to John the Ripper yielded an almost instant crack on the exact same hardware, proving that the industry-standard tool isn't always the best fit for available resources.

5. Conclusion

Before this room I could run individual recon tools, but I couldn't chain them into a full attack path on my own. Now I can: take a bare IP, work out it's a domain controller, pull account names out of it with no credentials at all, turn a Kerberos misconfiguration into real credentials, use those credentials to find more credentials, and finally turn a single over-privileged account into complete control of the domain. What I learned most was how much of the compromise didn't come from a clever exploit at all, it came from ordinary misconfigurations (an open null session, pre-authentication disabled on a service account, replication rights handed to a backup account) stacking on top of each other. Next time I'd spend more time up front checking my own machine's limits (RAM, GPU availability) before picking a cracking tool, rather than discovering the mismatch mid-attack. For a security analyst, this is exactly the mindset that matters on defense too: each individual

misconfiguration here looks minor in isolation, and the job is noticing how they chain together before an attacker does.